RECONCILE

Part 2: The Interface Layer

LLM-Driven Governance Interfaces from System Definitions

Credit IQ

March 2026

Thesis

AI commoditizes interfaces. Value lives in system architecture. This document specifies the layer where those two ideas meet: the system architecture generates the interface, and the AI renders it — under full governance constraint.

Companion to: Reconcile Technical Specification v5.0 (the kernel, primitives, graph foundations). This document assumes Part 1 as given and extends it with the interface layer, the LLM integration model, and the agentic UI architecture.

Table of Contents

(Generate in Word: References > Table of Contents)

I. The Insight

1. The Problem with Enterprise Interfaces

Every enterprise system has two implementations of its business rules: one in the backend (enforced) and one in the frontend (reimplemented). The backend says “this officer can approve loans up to 50L.” The frontend independently hides the approve button for loans over 50L. The two implementations drift. Bugs appear. Audit gaps emerge. Compliance failures follow.

This duplication exists because the interface layer has no access to the formal governance model. The backend enforces rules through code. The frontend guesses at those rules through separate code. They are two programs that must agree, maintained by two teams, with no shared source of truth.

2. What Reconcile Changes

The Reconcile kernel (Part 1, v5.0) makes the governance model machine-readable. The 9-tuple $\mathcal{R}$ = (S, E, T, P, R, V, I, D, G) is not documentation — it is an executable specification. Every valid transition, every role permission, every policy constraint, every invariant is queryable at runtime through the SystemContext API.

This means an interface layer does not need to reimplement governance rules. It can read them from the spec and derive the correct interface for any context: this resource, in this state, viewed by this role, under these policies. The rules exist once. Both enforcement and presentation derive from the same source.

3. The LLM Connection

In an agentic world, interfaces are increasingly generated by LLMs — conversational UIs, dynamic forms, contextual dashboards. The core problem with LLM-generated interfaces in regulated domains is trust: how do you ensure the AI doesn’t generate an interface that violates governance rules?

The answer: the system definition is the prompt. Not loosely — literally. The 9-tuple contains every constraint the LLM needs to generate a correct, policy-compliant, role-appropriate interface. The LLM doesn’t invent what buttons to show. The spec defines what transitions are valid. The LLM translates the spec into a rendered interface. The kernel validates every action the interface enables.

> **The Architecture**
>
> The governance rules exist once, formally, in the spec. The backend enforces them. The AI generates interfaces that comply with them. The kernel validates everything. The audit trail is complete.

II. Interface as Projection

4. The Projection Model

In Part 1, the graph layer is defined as a derived projection of the state store. The interface layer follows the same pattern: it is a derived projection of the system state through the role lens.

```
define_system()                          # the governance spec (source of
truth)
    │
    ├── Kernel                              # enforces transitions, policies,
invariants
    ├── Graph Projection                    # cross-resource analytics, GNN
    ├── Audit Layer                         # temporal graph, compliance
    └── Interface Projection (NEW)          # what the user sees and can do
            │
            ├── LLM reads spec + state + role
            ├── Generates contextual interface
            ├── User interacts
            ├── Actions routed to kernel
            └── Kernel validates and enforces
```

The interface layer is just another consumer of the SystemContext API — same as controllers, same as agents, same as the graph projection. It asks: “for this resource, in this state, viewed by this role, what’s valid?” And it renders the answer.

5. The Interface Context

For any given moment, the interface is fully determined by four inputs:

Input	Source	What It Determines
Resource state	State Store	What stage the entity is in; what data exists
Role permissions	Role definitions (R)	What actions are visible; what fields are accessible
Applicable policies	Policy engine (P)	What constraints apply; what warnings to surface
Agent proposals	Agent registry (I)	What AI-generated recommendations are available

These four inputs are already available through SystemContext. The interface layer queries them and computes the interface projection:

```
InterfaceProjection(resource_id, role) → {
```

```
    visible_fields: [field],            # derived from role permissions
    valid_actions: [transition],         # derived from state machine + role +
policies
    blocked_actions: [{action, reason}], # transitions that exist but are
policy-blocked
    warnings: [policy_message],          # applicable policy constraints to
surface
    proposals: [agent_proposal],         # AI recommendations with confidence
scores
    audit_summary: [recent_events],      # recent history for context
}
```

6. The Interface Projection API

6.1 Core Query

```
SystemContext.interface.project(resource_id, role) → InterfaceProjection
```

This is the single query that drives the entire interface layer. Everything a UI needs — what to show, what to enable, what to warn about, what to recommend — comes from this one call.

6.2 Batch Projection (List Views)

```
SystemContext.interface.project_list(resource_type, role, filters?) →
[InterfaceProjection]
```

For dashboards and list views: project interface state for multiple resources at once. Each resource gets its own projection based on its current state.

6.3 Action Execution

```
SystemContext.interface.execute_action(resource_id, action, actor, role) →
ActionResult
```

When the user takes an action, it routes through the kernel's normal transition pipeline. The interface layer never bypasses governance. ActionResult includes the new InterfaceProjection (the UI updates reactively after the transition).

6.4 Subscription (Real-Time Updates)

```
SystemContext.interface.subscribe(resource_id, role) →
Stream<InterfaceProjection>
```

For live interfaces: subscribe to projection changes. When the resource transitions (by any actor — human, controller, agent, system), the projection updates and the UI re-renders. This is the React analogy: state changes trigger re-renders, but the "state" is the governance context, not a JavaScript object.

III. LLM-Driven Interfaces

7. The System Definition as Prompt

The InterfaceProjection contains everything an LLM needs to generate a correct interface. The LLM's input is not a vague natural-language description of the business — it is a structured, machine-readable governance context.

7.1 What the LLM Receives

```
LLM Context = {
    resource: {type, state, desired_state, data},
    projection: {visible_fields, valid_actions, blocked_actions, warnings,
proposals},
    schema: {field definitions, types, constraints},
    history: [recent events for narrative context],
    output_modality: "form" | "conversation" | "dashboard" | "voice" | "api"
}
```

7.2 What the LLM Produces

The LLM generates an interface in the requested modality. Critically, the LLM cannot violate the governance spec because:

- The valid_actions list is computed by the kernel, not the LLM. The LLM can only present actions from this list.
- The visible_fields list is computed from role permissions. The LLM cannot expose fields outside this list.
- Every action the user takes is routed through the kernel's transition pipeline. Even if the LLM hallucinates an invalid option, the kernel rejects it.
- The audit trail records both the LLM's generated interface and the user's actions. Full traceability.

8. Interface Modalities

The same InterfaceProjection can drive any interface modality. The governance layer is modality-agnostic.

Modality	How the LLM Uses the Projection	Example
Web form	Renders fields from visible_fields, buttons from valid_actions, alerts from warnings	Loan approval form with policy warnings
Conversation	Narrates the resource state, offers valid_actions as choices, explains blocked_actions	"This loan needs committee approval. Shall I escalate?"

Modality	How the LLM Uses the Projection	Example
Dashboard	Aggregates projections across resources, highlights proposals	Officer's portfolio view with risk-flagged loans
Voice	Reads state, offers actions audibly, confirms before executing	Voice-guided audit walkthrough
API response	Returns projection as JSON for programmatic consumers	Third-party system integration
Mobile	Optimized touch interface from the same projection	Field officer's loan inspection app

9. Conversational Workflow Example

A loan officer interacts with the system through a conversational interface driven by the governance spec:

Officer: "Show me loan LOAN-4782."

LLM reads InterfaceProjection for LOAN-4782 with role=loan_officer:

```
  state: UNDERWRITING
  valid_actions: [APPROVE, REJECT, REQUEST_DOCUMENTS]
  blocked_actions: [{APPROVE, "Amount ₹80L exceeds officer limit of ₹50L"}]
  warnings: ["High-value loan: committee approval required"]
  proposals: [{agent: "risk-scoring", score: 0.73, recommendation:
"conditional_approve"}]
```

LLM: "Loan LOAN-4782 is in underwriting. Amount is ₹80L for Acme Corp. The risk agent scored it at 0.73 and recommends conditional approval. However, this exceeds your approval limit of ₹50L — it needs committee approval. Would you like to escalate to committee, reject, or request additional documents?"

Officer: "Escalate to committee with the risk agent's recommendation."

LLM calls: SystemContext.interface.execute_action("LOAN-4782", "ESCALATE_TO_COMMITTEE", actor="officer-7", role="loan_officer")

Kernel validates, transitions, emits events, records audit.

LLM: "Done. LOAN-4782 has been escalated to committee. The risk score and your recommendation are attached to the audit record. Committee will see this in their queue."

Every sentence the LLM produced was derived from the InterfaceProjection. The blocked_actions list told it the officer couldn't approve directly. The proposals list told it about the risk score. The valid_actions list told it what options to present. The kernel handled the actual transition. The audit trail is complete.

10. The Governance Guarantee

> **The Invariant**
>
> No interface — whether generated by an LLM, a React component, a mobile app, or a voice agent — can cause a governance violation. The kernel validates every transition independently of the interface layer. The interface proposes; the kernel decides. This is "agents propose, system decides" applied to the interface itself.

10.1 Defense in Depth

The system enforces governance at three layers:

Layer	What It Prevents	Mechanism
Interface projection	Showing invalid options	valid_actions computed from state machine + role + policies
Kernel validation	Executing invalid transitions	Full transition pipeline: state check, role check, guards, policies, invariants
Audit recording	Hiding what happened	Every action, including LLM-generated interfaces, logged with authority_level

Even if the LLM hallucinates an action, Layer 2 catches it. Even if Layer 2 has a bug, Layer 3 records the attempt. This is defense in depth for AI-generated governance interfaces.

10.2 Authority Level: INTERFACE

Part 1 defines four authority levels: HUMAN, CONTROLLER, AGENT, SYSTEM. The interface layer adds a fifth:

Level	Source	Audit Meaning
HUMAN	Direct API/UI action	User explicitly chose this action
INTERFACE	LLM-generated interface action	User confirmed an LLM-presented option
CONTROLLER	Reactive/reconciliation controller	System automation
AGENT	AI proposal accepted	Agent recommendation accepted
SYSTEM	Internal kernel operation	Timeout, auto-escalation, etc.

The INTERFACE authority level distinguishes "the user clicked a button they found themselves" from "the user confirmed an option the LLM presented." This matters for audit: regulators may want to know whether decisions were influenced by AI-generated interfaces.

IV. Architecture

11. Component Architecture

```
define_system()
       │
       ├── Kernel (Part 1)
       │     State Store, Event Log, Policy Engine,
       │     Invariant Checker, Controller Scheduler, Audit Log
       │
       ├── Graph Projection (Part 1, §VII)
       │     Schema Graph, Instance Graph, Temporal Graph
       │
       └── Interface Layer (Part 2 — this document)
             │
             ├── Interface Projection Engine
             │     Computes: resource + role → what to show/enable
             │
             ├── LLM Adapter
             │     Translates projections into modality-specific interfaces
             │
             ├── Modality Renderers
             │     Web (React/HTML), Conversation, Voice, API, Mobile
             │
             └── Action Router
                   Routes user actions back to kernel transition pipeline
```

12. Interface Projection Engine

The projection engine is a pure function: given a resource and a role, it computes the InterfaceProjection by querying the kernel's existing APIs.

12.1 Computation Steps

1. Fetch resource from state store (current state, data, desired state)
2. Fetch valid transitions from state machine for current state
3. Filter transitions by role permissions (R)
4. Evaluate policies (P) against each remaining transition to classify as valid or blocked

5. Fetch agent proposals for this resource
6. Fetch recent events for context
7. Assemble InterfaceProjection

This is a read-only operation. It queries the kernel but does not modify state. It can be cached and invalidated on state change events.

12.2 Caching Strategy

InterfaceProjections are invalidated when:

- The resource transitions (state change event)
- A policy is updated (policy change event)
- An agent emits a new proposal for the resource
- The role definition changes (system configuration event)

Between invalidation events, the projection is stable and can be served from cache. For most resources, the projection changes only on transitions — which happen at human timescales (minutes to days), not machine timescales.

13. LLM Adapter

The LLM adapter translates InterfaceProjections into natural-language or structured UI outputs. It is a stateless function:

```
LLMAdapter.render(projection, modality, user_message?) → interface_output
```

13.1 System Prompt Construction

The LLM's system prompt is constructed from the projection:

```
system_prompt = f"""
You are an interface for a {resource.type} governance system.
The resource is in state {resource.state}.

VALID ACTIONS the user can take:
{format(projection.valid_actions)}

BLOCKED ACTIONS (explain why if asked):
{format(projection.blocked_actions)}

WARNINGS to surface:
{format(projection.warnings)}

AI RECOMMENDATIONS:
```

```
{format(projection.proposals)}


RULES:
- Only offer actions from the VALID ACTIONS list
- If the user asks for a blocked action, explain the reason
- When the user confirms an action, call execute_action
- Never fabricate data not present in the resource
"""
```

The LLM does not need to understand banking, insurance, or healthcare. It needs to understand the projection schema — which is domain-agnostic. The domain knowledge lives in the system definition, not in the LLM prompt.

13.2 Tool Binding

The LLM is given tools that map to kernel operations:

```
tools = [
    execute_action(resource_id, action)    # → kernel transition
    get_resource_details(resource_id)      # → read from state store
    get_audit_trail(resource_id)           # → read from audit log
    get_causal_chain(event_id)             # → temporal graph traversal
    list_resources(type, filters)           # → list view
]
```

Every tool routes through the kernel. The LLM cannot bypass governance.

14. Modality Renderers

14.1 React/Web Renderer

Generates React components from the projection:

```
function GovernedView({ resourceId, role }) {
    const projection = useProjection(resourceId, role);
    return (
        <ResourceCard
            fields={projection.visible_fields}
            actions={projection.valid_actions}
            warnings={projection.warnings}
            proposals={projection.proposals}
            onAction={(action) => executeAction(resourceId, action, role)}
```

```
        />
    );
}
```

The component renders only what the projection allows. Role-based visibility, policy warnings, and valid actions are automatic — the developer never hand-codes them.

14.2 Conversational Renderer

Wraps the LLM adapter for chat-based interaction. See Section 9 for the worked example.

14.3 API Renderer

Returns the InterfaceProjection as JSON. Third-party systems consume this to build their own interfaces while respecting governance constraints.

```
GET /api/interface/{resource_type}/{id}?role={role}


Response: {
    resource: {id, type, state, data},
    valid_actions: ["APPROVE", "REJECT", "ESCALATE"],
    blocked_actions: [{action: "DISBURSE", reason: "Not yet approved"}],
    warnings: ["High-value: committee required"],
    proposals: [{agent: "risk", score: 0.73, action: "conditional_approve"}]
}
```

V. Generality: Beyond Banking

15. The Universal Enterprise Pattern

The interface projection model is not specific to banking. It applies to any enterprise system where decisions must be governed. The 9-tuple $\mathcal{R}$ = (S, E, T, P, R, V, I, D, G) describes virtually every enterprise workflow:

Domain	States	Roles	Key Policies	Interface Implication
Procurement	Requested → Approved → PO Issued → Received → Paid	Requester, Approver, Finance, Vendor	Budget limits, approval chains	Approver sees approve/reject; requester sees status only
HR Onboarding	Offer → Accepted → Background → Provisioning → Active	Recruiter, HR, IT, Manager, Employee	Compliance docs, system access policies	IT sees provisioning actions; employee sees welcome flow
Clinical Trials	Screening → Enrolled → Treatment → Follow-up → Complete	PI, CRA, IRB, Sponsor, Patient	Regulatory (FDA/EMA), protocol deviations	IRB sees safety data; PI sees treatment actions
Legal Matter	Intake → Review → Active → Discovery → Trial → Closed	Partner, Associate, Paralegal, Client	Privilege, conflicts of interest	Associate sees research actions; client sees status
Supply Chain	Order → Manufactured → Shipped → Customs → Delivered	Buyer, Supplier, Logistics, Customs	Insurance coverage, expiry constraints	Customs sees clearance actions; buyer sees tracking

In every case, the interface projection is the same computation: given this entity in this state, viewed by this role, what's visible and what's actionable? The domain changes. The pattern does not.

16. The 50-Line Enterprise System

The ultimate developer experience: describe the entire governance model in ~50 lines, and get backend enforcement + LLM-driven interfaces + audit trails + graph analytics automatically.

```
system = define_system(
    name="procurement",
    states=["REQUESTED","APPROVED","PO_ISSUED","RECEIVED","PAID"],
    transitions=[
        ("REQUESTED","APPROVED"), ("APPROVED","PO_ISSUED"),
        ("PO_ISSUED","RECEIVED"), ("RECEIVED","PAID"),
```

```
        ("REQUESTED","REJECTED"),
    ],
    roles={
        "requester": ["view", "transition:REQUESTED"],
        "approver": ["view", "transition:APPROVED", "transition:REJECTED"],
        "finance":  ["view", "transition:PO_ISSUED", "transition:PAID"],
    },
    policies=[
        Policy("budget_limit", evaluate=lambda r, ctx:
            PolicyResult(passed=r.data["amount"] <=
dept_budget(r.data["dept"]),
                         message="Exceeds department budget")),
            applicable_states=("REQUESTED",)),
    ],
    invariants=[
        Invariant("positive_amount", mode="strong",
            check=lambda r, ctx: InvariantResult(holds=r.data["amount"] > 0)),
    ],
)
```

From this definition, the system generates:

- Backend: state machine enforcement, RBAC, policy evaluation, audit trails
- Graph: cross-resource analytics, relationship tracking
- Interface: LLM-driven conversational UI, React dashboards, API projections
- All governed by the same spec. All auditable. All consistent.

17. The Competitive Landscape

Every other approach to AI-powered enterprise software falls into one of three categories:

Approach	What It Does	Why It Fails
LLMs generating CRUD apps	AI writes code for basic applications	Commoditized in months; no governance; no audit
LLMs as chatbot wrappers	AI bolted onto existing systems	No formal constraint; hallucination risk; governance gaps
Low-code + AI	Visual builders with AI assistance	Still manually coded rules; AI is cosmetic; rules drift

Reconcile + LLM-driven interfaces is none of these. The governance rules exist once, formally. The AI generates interfaces within those constraints. The kernel validates everything. No other system in the market unifies backend governance and frontend generation through the same formal model.

VI. DSL Extensions

18. Interface Configuration in define_system()

The interface layer is configured as part of the system definition:

```
system = define_system(
    name="loan",
    states=[...], transitions=[...], roles={...}, policies=[...],

    interface={
        "modalities": ["web", "conversation", "api"],
        "field_labels": {"amount": "Loan Amount (₹)", "applicant_id": "Applicant"},
        "action_labels": {"APPROVE": "Approve Loan", "REJECT": "Reject Application"},
        "views": {
            "officer_dashboard": {"role": "loan_officer", "list": True, "default_filters": {"state": "UNDERWRITING"}},
            "committee_queue": {"role": "committee", "list": True, "default_filters": {"state": "ESCALATED"}},
        }
    }
)
```

19. View Definitions

```
define_view("loan_officer_dashboard",
    resource="loan",
    role="loan_officer",
    layout="list",
    columns=["id", "applicant", "amount", "state", "risk_score"],
    default_sort="created_at",
    highlight_when={"risk_score > 0.8": "warning", "state == ESCALATED": "info"},
)
```

20. Conversational Flow Configuration

```
define_conversation_flow("loan_processing",
```

```
    resource="loan",
    greeting="I can help you process loans. Which loan would you like to
review?",
    on_blocked_action="explain_policy",     # LLM explains which policy
blocked and why
    on_agent_proposal="present_with_context", # LLM narrates agent reasoning
    confirmation_required=["APPROVE", "REJECT", "DISBURSE"],  # double-confirm
critical actions
)
```

VII. Implementation Roadmap

21. Phase Mapping

The interface layer maps to Phases 4-5 of the Part 1 roadmap:

Phase	What	Depends On
Phase 2 (Part 1)	Kernel: invariants, desired state, Postgres	Core kernel complete
Phase 3 (Part 1)	Graph projections, REST API, WebSocket	State store + event log stable
Phase 4 (this doc)	Interface Projection Engine + API renderer	REST API + role/policy queries
Phase 5 (this doc)	LLM Adapter + Conversational renderer	Projection engine + LLM integration
Phase 6 (this doc)	React renderer + View DSL + real-time subscriptions	Projection engine + WebSocket

22. Phase 4: Interface Projection Engine

Target: ~1,000-1,500 LOC

- InterfaceProjection data structure
- Projection computation (resource + role → projection)
- API endpoint: GET /api/interface/{type}/{id}?role={role}
- Projection caching with event-driven invalidation
- INTERFACE authority level in audit records

23. Phase 5: LLM Adapter

Target: ~800-1,200 LOC

- System prompt construction from InterfaceProjection
- Tool binding (execute_action, get_resource, get_audit)
- Conversational flow engine
- Multi-LLM support (Claude, GPT, open-source models)
- Audit logging of LLM-generated interfaces

24. Phase 6: Renderers + DSL

Target: ~1,500-2,000 LOC

- React component library (GovernedView, ActionButton, PolicyWarning)
- View DSL (define_view, define_conversation_flow)

- Real-time subscription via WebSocket
- Interface configuration in define_system()

VIII. Formal Properties

25. The Interface Invariant

The interface layer satisfies a formal invariant:

> **Interface Safety Theorem**
>
> ∀ interface I generated from projection P: ∀ action A presented by I: A ∈ valid_actions(P). No interface, regardless of modality or generation method, can present an action that is not in the kernel-computed valid_actions set.

This holds because:

8. The projection is computed by the kernel, not the interface layer
9. The LLM's system prompt constrains it to the valid_actions list
10. Even if the LLM violates its prompt, the kernel rejects invalid transitions
11. The audit trail records all attempts, including rejected ones

26. The Reconciliation Functor

React, Kubernetes, and Reconcile are all instances of the same abstract pattern: a functor from desired state to observed state, with reconciliation as the natural transformation.

System	Source	Target	Functor	Reconciliation
React	Component state	DOM	render()	Virtual DOM diff + patch
Kubernetes	Cluster spec	Infrastructure	Controller loops	Desired state → actual state
Reconcile (kernel)	System definition	Institutional state	Transition pipeline	current_state → desired_state
Reconcile (interface)	System definition	User interface	Projection engine	governance spec → UI

The interface layer is a second functor from the same source (system definition) to a different target (user interface). Both functors — the kernel functor and the interface functor — share the same system definition as input. This is why they are guaranteed to be consistent: they derive from the same source of truth.

27. The Full Architecture Statement

> **Architecture Statement (Complete)**
>
> "This system coordinates [X states] across [Y roles] under [Z policies] maintaining [W invariants] reconciling toward [desired states] over [T time horizon], with AI at [N decision nodes] and governance-constrained interfaces in [M modalities] — all from a single system definition."

AI commoditizes interfaces. Value lives in system architecture.
The system architecture generates the interface.
The AI renders it. The kernel governs it.